Yes—**use `fixup` and `autosquash primarily on branches whose history you're allowed to rewrite**, usually your own feature branch.

Practical rule

Branch type	Fixup + autosquash?	Why
Your local feature branch	Yes	Safest and most useful
Your pushed personal feature branch	Usually yes	Push afterward with `--force-with-lease`
Shared feature branch	Usually no	Rewriting disrupts teammates
`main`, `develop`, release branches	No	Published history should remain stable
Another developer's branch	No, unless coordinated	Their local history may depend on it

The important distinction is not strictly **local versus remote**. It is **private versus shared history**.

A remote branch can still be effectively private when only you work on it.

A modern feature-branch workflow

During development, create logical commits:

```bash
git add src/login.ts
git commit -m "Add login validation"

git add tests/login.test.ts
git commit -m "Add login validation tests"
```

When you later correct something belonging to an earlier commit:

```bash
git add src/login.ts
git commit --fixup <target-commit>
```

Before requesting final review or merging:

```bash
git fetch origin
git rebase -i --autosquash origin/main
```

Then run your tests and push:

```bash
git push --force-with-lease
```

This produces a clean PR history without forcing you to obsess over perfect commits while actively developing.

During review

Fixup commits are particularly useful for PR feedback.

Suppose a reviewer asks you to correct the validation logic. Instead of modifying the old commit immediately, make the correction transparently:

```bash
git commit --fixup <validation-commit>
git push
```

The reviewer can clearly see the new fix in the PR.

Once the review is nearly complete, autosquash it:

```bash
git rebase -i --autosquash origin/main
git push --force-with-lease
```

There are two common team styles here:

Keep fixups until approval

You push `fixup!` commits while the PR is being reviewed, then autosquash once the review is complete.

Advantages:

- Reviewers easily see what changed since their last review.
- You avoid repeatedly rewriting the PR history.
- Fewer force-pushes during active review.

This is often the smoother workflow.

Autosquash continuously

You autosquash every correction immediately and force-push the branch.

Advantages:

- The PR history always looks clean.

Disadvantages:

- Commit hashes change repeatedly.
- Review comments and comparisons may become harder to follow.
- Teammates must constantly resynchronize.

For most teams, I would avoid compulsively rebasing after every tiny correction. A PR does not need museum-quality history at every moment.

Shared feature branches

Suppose you and another developer both work on:

```
feature/payment
```

You autosquash and force-push it. Their local branch still refers to the old commits:

```
Remote after rebase: A---B'---C'
Teammate's branch:    A---B----C---D
```

Now they must reconcile the rewritten history, potentially through:

```
Bash

git fetch origin
git rebase --onto origin/feature/payment <old-base> their-work
```

That is inconvenient and error-prone.

On shared branches, prefer adding normal commits:

```
Bash

git commit -m "Fix payment validation"
git push
```

Then either:

- let the platform squash everything when merging the PR, or
- coordinate one final history cleanup when nobody else is working on the branch.

Consider your repository's merge strategy

Modern teams commonly use one of these approaches:

Squash merge

The PR platform turns the entire feature branch into one commit on `main`.

```
Feature commits:
A---B---C---D

After squash merge:
main---S
```

In this workflow, autosquashing your feature branch is optional. The destination history will be clean anyway.

Still, clean feature commits can help reviewers understand the work, especially for large PRs.

Rebase and merge

Every feature commit is replayed onto `main`.

```
main---A'---B'---C'
```

Here, commit quality matters much more. Autosquash is valuable because all your commits will remain visible on `main`.

Merge commit

The feature branch is merged with an explicit merge commit.

```
main-------M
          /
      A---B---C
```

The complete feature history remains visible, so cleaning noisy fixup commits may still be worthwhile.

What I recommend

For a personal feature branch:

```bash
# Work normally
git commit -m "Implement notification handling"

# Add corrections to the appropriate existing commit
git commit --fixup <target-commit>

# Before final approval or merging
git fetch origin
git rebase -i --autosquash origin/main

# Validate the rewritten branch
npm test  # or your project's test command

# Update the remote branch safely
git push --force-with-lease
```

For a shared branch:

```bash
git fetch origin
git merge origin/main
git push
```

Or use normal correction commits and rely on squash merge.

Team conventions worth adopting

A healthy team policy might be:

> Developers may rebase and autosquash personal feature branches. Do not rewrite `main`, release branches, or shared feature branches. Use `--force-with-lease` for rewritten remote branches. Avoid force-pushing while another developer is actively using the branch.

You can also enable autosquash globally:

```bash
git config --global rebase.autosquash true
```

Then:

```bash
git rebase -i origin/main
```

automatically recognizes `fixup!`, `squash!`, and `amend!` commits.

My default recommendation is: **develop freely, use fixup commits for review corrections, autosquash near the end, and rewrite only branches you own.**

Could you also explain about cherry pick?

`git cherry-pick` copies one or more specific commits from another branch onto your current branch.

It is useful when you want **particular changes**, not the entire branch history.

Basic idea

Suppose the history looks like this:

```
main:       A---B---C
                 \
feature:          D---E---F
```

You are on `main`, and only commit `E` contains a bug fix you need.

```
Bash

git switch main
git cherry-pick <commit-E-hash>
```

Git creates a new commit on `main`:

```
main:       A---B---C---E'
                 \
feature:          D---E---F
```

`E'` has the same changes as `E`, but it is a **new commit with a different hash** because it has a different parent.

Typical usage

First find the commit:

```
Bash
```

```
git log --oneline feature/payment
```

Example output:

```
8fd21ab Add payment screen
31ac879 Fix payment timeout
ad110e2 Add payment analytics
```

Apply only the timeout fix:

```
Bash

git switch main
git cherry-pick 31ac879
```

Cherry-pick multiple commits

You can cherry-pick several named commits:

```
Bash

git cherry-pick 31ac879 ad110e2
```

Git applies them in the order you provide.

For a continuous range:

```
Bash

git cherry-pick 31ac879^..ad110e2
```

The `^` includes the first commit in the range.

Without it:

```
Bash

git cherry-pick 31ac879..ad110e2
```

Git excludes `31ac879`.

A safer, clearer approach is often to list the commits explicitly unless the range is obvious.

What happens during a conflict

If the target branch has conflicting changes, Git pauses:

```
CONFLICT (content): Merge conflict in src/payment.ts
```

Resolve the file, then:

```bash
git add src/payment.ts
git cherry-pick --continue
```

To cancel the entire operation:

```bash
git cherry-pick --abort
```

To skip the current commit:

```bash
git cherry-pick --skip
```

Skipping is mainly useful when cherry-picking a sequence and one commit is no longer needed or is already effectively present.

When cherry-pick is a good choice

Backporting a bug fix

A bug was fixed on `main`, but a release branch also needs it:

```bash
git switch release/2.4
git cherry-pick <bug-fix-commit>
```

This is one of the most common legitimate uses.

Moving a commit made on the wrong branch

You accidentally committed on `main` instead of your feature branch:

```bash
git switch feature/login
git cherry-pick <accidental-commit>
```

Then remove it from local `main`, assuming it has not been shared:

```bash
git switch main
git reset --hard origin/main
```

Be careful with `reset --hard`, because it discards uncommitted changes and local commits.

Taking one useful change from an abandoned branch

A large experimental branch contains one reusable fix:

```bash
git switch feature/new-ui
git cherry-pick <useful-fix>
```

Applying the same fix to several maintained versions

```bash
git switch release/1.x
git cherry-pick <fix>

git switch release/2.x
git cherry-pick <fix>
```

This is common in release maintenance.

When not to use cherry-pick

Do not use cherry-pick just to keep a feature branch synchronized with `main`.

For that, use:

```bash
git fetch origin
git rebase origin/main
```

or:

```bash
git fetch origin
git merge origin/main
```

Cherry-picking many commits from one branch often means you probably wanted to merge or rebase the branch instead.

For example, this is suspicious:

```bash
git cherry-pick A
git cherry-pick B
git cherry-pick C
git cherry-pick D
git cherry-pick E
```

If those commits represent one complete feature, merge or rebase is usually more appropriate.

Cherry-pick versus merge

Merge

```bash
git merge feature/payment
```

Meaning:

> Bring the branch and its history into my current branch.

It preserves the relationship between the branches.

Cherry-pick

```bash
git cherry-pick <commit>
```

Meaning:

> Copy this specific commit’s changes onto my current branch.

It does not preserve the branch relationship.

Cherry-pick versus rebase

Rebase moves or replays a series of commits onto another base:

```bash
```

```
git rebase origin/main
```

Cherry-pick manually copies selected commits:

```
Bash

git cherry-pick <commit>
```

Internally, rebase behaves somewhat like a coordinated sequence of cherry-picks, but the intent is different:

- **Rebase:** reposition this branch's history.
- **Cherry-pick:** selectively copy commits elsewhere.

Cherry-pick and duplicate commits

Because cherry-pick creates a new commit, Git may later contain two commits with equivalent changes:

```
feature:  D---E
main:     A---B---E'
```

If the feature branch is later merged into `main`, Git usually recognizes the resulting file state, but the history can become confusing. You may see both `E` and `E'` in different places.

This is why cherry-picking should be selective and intentional rather than your default integration strategy.

Useful options

Edit the commit message

```
Bash

git cherry-pick -e <commit>
```

Git opens the commit message editor before creating the new commit.

Apply changes without committing

```
Bash

git cherry-pick --no-commit <commit>
```

Or:

```bash
git cherry-pick -n <commit>
```

This applies the changes to your working tree and staging area but does not create a commit.

Useful when combining several commits into one:

```bash
git cherry-pick -n <commit-1>
git cherry-pick -n <commit-2>
git commit -m "Backport payment stability fixes"
```

Add source information

```bash
git cherry-pick -x <commit>
```

This appends a line such as:

```
(cherry picked from commit 31ac879...)
```

to the commit message.

This is especially useful for backports to release branches because it records where the fix came from.

A common release workflow is:

```bash
git switch release/2.4
git cherry-pick -x <fix-commit>
```

Sign off the commit

```bash
git cherry-pick -s <commit>
```

This adds a `Signed-off-by` line. It is relevant only in projects that use Developer Certificate of Origin workflows.

Cherry-picking a merge commit

A merge commit has more than one parent, so Git needs to know which parent should be treated as the mainline:

```bash
git cherry-pick -m 1 <merge-commit>
```

`-m 1` normally means use the first parent as the mainline.

This can be tricky and error-prone. It is often better to cherry-pick the actual individual fix commit rather than the merge commit itself.

Practical examples

Backport a production fix

The fix exists on `main`:

```bash
git switch main
git log --oneline
```

You find:

```
91bd4f2 Fix crash when restoring session
```

Apply it to the release branch:

```bash
git switch release/3.2
git pull --ff-only
git cherry-pick -x 91bd4f2
git push
```

Move an accidental commit

You committed on the wrong branch:

```bash
git log --oneline -1
```

Output:

```
c7a28f1 Add login timeout handling
```

Move it:

```bash
git switch feature/login-timeout
git cherry-pick c7a28f1
```

Then clean your local `main`, provided the commit was not pushed:

```bash
git switch main
git reset --hard HEAD~1
```

Take only part of another feature

```bash
git switch feature/notifications
git cherry-pick <shared-utility-commit>
```

This is reasonable when the commit is self-contained and does not depend on earlier commits.

Dependency warning

A commit may depend on earlier changes.

For example:

```
D Add payment service
E Add retry logic using payment service
```

Cherry-picking only `E` may fail to compile because `D` is missing.

Before cherry-picking, inspect the commit:

```bash
git show <commit>
```

And inspect its surrounding history:

```bash
```

```
git log --oneline --graph <branch>
```

Sometimes you need both commits:

```
Bash

git cherry-pick <D> <E>
```

Best practice in modern team workflows

Use cherry-pick mainly for:

- Backporting fixes to release branches.
- Recovering commits made on the wrong branch.
- Reusing a small, independent change.
- Applying an urgent fix across maintained branches.

Avoid using it for:

- Regularly syncing feature branches with `main`.
- Moving an entire feature branch.
- Copying long chains of dependent commits.
- Bypassing the normal PR review process.

A solid team convention is:

> Use merge or rebase for branch integration. Use cherry-pick only for intentional, selective commit transfer. Use `-x` when backporting fixes so the source commit remains traceable.

One compact mental model:

```
merge       = bring the branch
rebase      = relocate the branch
cherry-pick = copy selected commits
fixup       = repair an earlier commit
autosquash  = fold those repairs into their targets
```

when I cherry-picked a commit from main to my branch, what happens when I merge my pr to main?

Suppose `main` already contains commit `M`, and you cherry-pick it onto your feature branch:

```
main:    A---B---M
              \
feature:       M'---F
```

`M'` contains roughly the same changes as `M`, but it has a different commit hash.

When you merge the PR back into `main`, Git compares your branch against the common ancestor. Since the changes from `M` are already present on `main`, Git normally **does not apply those changes a second time**. Only your genuinely new feature changes, such as `F`, are added.

Conceptually, the result is:

```
main: A---B---M---------Merge
             \         /
              M'---F
```

The history may still contain both `M` and `M'`, but the file contents are not normally duplicated.

What happens by merge strategy

Merge commit

Git preserves your branch history, so the cherry-picked commit `M'` may remain visible:

```
A---B---M-----------X
        \         /
         M'---F---
```

This works, but the history can look confusing because `M` and `M'` represent equivalent changes.

Squash merge

The platform combines the net PR changes into one new commit.

Because `M` is already on `main`, the squash usually contains only the changes introduced by `F`:

```
A---B---M---S
```

This is generally the least confusing result.

Rebase and merge

The platform replays your feature commits onto the current `main`.

When Git reaches `M'`, it may detect that the patch is already present and skip it:

```
warning: skipped previously applied commit ...
```

The resulting history is usually:

```
A---B---M---F'
```

However, patch detection is not perfect. If `M'` was modified, conflict-resolved differently, or combined with other changes, Git may not recognize it as equivalent.

Possible conflict scenario

Suppose you cherry-pick `M`, then modify the same code in your feature commit:

```
M'  copied from main
F   changes the same lines further
```

When merging, Git still knows `M` is already on `main`, but it must integrate the additional changes from `F`. That can merge cleanly or produce conflicts depending on what changed on `main` afterward.

Best practice

If your branch was originally created from an old version of `main`, and you need all recent `main` changes, do not cherry-pick individual commits. Rebase or merge `main` instead:

```
Bash

git fetch origin
git rebase origin/main
```

or:

```
Bash

git fetch origin
```

Use cherry-pick when you intentionally need only one isolated commit.

If you already cherry-picked a commit from `main`, it is usually safe to merge the PR. The main downside is **messier or duplicated-looking history**, not duplicated code.

Before merging, you can clean the branch by rebasing onto current `main`:

```bash
git fetch origin
git rebase origin/main
```

Git will often skip the already-applied cherry-picked commit automatically. Then verify:

```bash
git log --oneline --graph
git diff origin/main...HEAD
```

The most important command is:

```bash
git diff origin/main...HEAD
```

It shows the actual net changes your PR will introduce, regardless of the confusing commit history.